

IAM Hardening Summary - Northstar Assist

Tarie Nosworthy

18 September 2026

Contents

Northstar Assist — Least-Privilege Access Controls	1
0. Status — the logging regression, and how it was closed	2
0.1 What was wrong	2
0.2 Root cause	2
0.3 What was done about it	2
0.4 Result	3
1. Method	3
2. Roles reviewed	4
A finding about where the permissions actually live	4
3. Changes applied	4
3.1 Harness execution role	5
3.2 Gateway service role — 2 bare wildcards to 0	6
3.3 Knowledge base service role	7
4. Surviving wildcards — every one accounted for	7
4.1 The account-field wildcard, stated plainly	10
4.2 Wildcards removed rather than justified	10
4.3 One wildcard justified by shape, not by experiment	10
4.4 Reading a removal result honestly	11
5. Verification — two kinds, and why one was not enough	11
5.1 Behavioural verification	11
5.2 Why that was not enough	11
5.3 Policy-level verification	11
5.4 Reproducing this without AWS	12
6. What an attacker gains — before versus after	12
7. Remaining recommendations	12
Items 6-9, raised after the first review — now closed	13
Still open	13
Evidence	14

Northstar Assist — Least-Privilege Access Controls

Assessment date: 18 September 2026 **Rebuilt and re-verified:** 19 September 2026 **Author:** Tarie Nosworthy **Environment:** AWS account 118924230273, us-east-1

The work was carried out over two Cloud Lab sessions in the same account. The second rebuilt the environment to re-apply the corrected hardening and capture a verified after-state (\$0). Resource identifiers therefore differ between the two sets of evidence — the knowledge base is MHJCWFFIBH in the first and V2RJQEGEIW in the second, and the service roles carry different generated suffixes — while the account, region, bucket name and configuration are identical.

Every policy document quoted here was **read from the live account**, narrowed programmatically, applied, and verified. Nothing was retyped. The complete before/after documents and unified diffs are in `submission/iam/`, the machine-readable change log in `evidence/iam/iam_changes.json`, and the generator's own output in `evidence/iam/generated-change-log.md`.

0. Status — the logging regression, and how it was closed

First submission (18 September 2026): defective. Rebuilt and re-verified (19 September 2026): clean. This section is kept in full because the defect is part of the record, not because it is still open.

0.1 What was wrong

An external review found that the hardening pass had **widened** three CloudWatch Logs statements rather than narrowing them:

Statement	Before	After (as applied)	Effect
CloudWatchLogsGroup	...:log-group:/aws/bedrock-agentcore/runtimes/*	...:log-group:/aws/bedrock-agentcore/*	widened
CloudWatchLogsStream	.../runtimes/*:log-stream:*	.../aws/bedrock-agentcore/*	widened
CloudWatchLogsPutResourcePolicy	.../runtimes/harness_NorthstarAssist-*	.../aws/bedrock-agentcore/*	widened
CloudWatchLogsDescribeGroups	...:log-group:*	the narrowed pair	narrowed (correct)

All four additionally gained `...:log-group:/northstar-assist/model-invocations:*` — giving the agent’s own execution role **write access to the log group recording its own model invocations**. Threat **R-02** was marked *mitigated* on the strength of this work. It was not mitigated; it was made worse.

A **fourth** widening, in no review, was then found by the policy auditor built to check this class of defect: `kb/S3GetObjectStatement` went from `bucket/*` to `bucket + bucket/*`. The console’s knowledge-base S3 policy was already textbook least-privilege — `ListBucket` on the bucket, `GetObject` on the objects, both under an `aws:ResourceAccount` condition — and the hardening pass made it broader. It grants nothing exploitable, since `GetObject` on a bucket ARN matches no object, but it is the same fault.

0.2 Root cause

`resolve_targets()` returned one ARN list per service for the whole role, and `narrow_policy()` applied it to every statement of that service, discarding each statement’s own tighter scope. The deeper fault is that **the narrowing pass never verified it had narrowed** — it computed a replacement resource set and wrote it without comparing it to the original.

Two further faults let it go unnoticed:

- **--verify verified the wrong thing.** It confirmed the agent still answered using retrieval. That is necessary and completely blind to a widening, because widening a permission never breaks anything. Behavioural verification cannot detect a permission change.
- **The wildcard counter matched only bare Resource: "*".** Seven partial wildcards were therefore never counted, and the summary claimed the gateway role had reached zero.

0.3 What was done about it

1. **is_narrower()** guard in `scripts/20_harden_iam.py`. Every proposed ARN must be covered by at least one original ARN, or the statement is left exactly as the console wrote it and flagged **KEPT-WOULD-WIDEN**. Leaving a permission too broad is recoverable; silently broadening one is not.
2. **scripts/22_iam_audit.py**, a policy-level auditor. It compares every statement’s before and after, fails on any widening that is not explicitly declared, and fails on any surviving wildcard with no entry in `iam/wildcard-register.json`. A missing entry is a failure, not a default pass.
3. **tests/test_no_widening.py**, an offline regression test needing no AWS credentials. It replays the captured console policies through the real narrowing pass and asserts nothing widens: **16 policies, 74 statements, 0 widened** — both sessions’ captures.

4. **A rebuild.** The environment was recreated on 19 September, the corrected hardening applied, and the result verified against live IAM.

0.4 Result

iam/iam_after.json, produced by reading the deployed policies back from IAM:

	First submission	Rebuild
Statements widened	4 (undetected)	0
Statements removed	9	14
Surviving wildcards	22 (7 partial uncounted)	17
Wildcards without justification	7	0
Audit verdict	FAIL	PASS

The first submission’s evidence is preserved unedited in iam/session1-2026-09-18/. It has deliberately not been retouched — a before/after evidence set that has been corrected after the fact is worth nothing — and 22_iam_audit.py --expect-fail pointed at that directory exits 0 *because* it fails, which is how the detector demonstrates it detects the real thing rather than asserting that it would.

1. Method

The policies were narrowed, not replaced.

The obvious approach — write ideal policies from scratch and overwrite the console’s — has two problems. It requires inventing IAM action names, some of which do not exist. And it produces no honest before/after comparison, because the “before” becomes a description rather than a document.

Instead scripts/20_harden_iam.py reads what the console actually attached and applies these transformations:

Transformation	Rule
NARROWED	A statement granting an action on a broad resource is rewritten to the specific ARNs this system uses, resolved from live state
SPLIT	A statement whose actions span several resource groups is divided so each carries its own scope
REMOVED	A statement whose actions serve only a feature that was never configured is deleted entirely
ACTION-REMOVED	An individually indefensible action is stripped from an otherwise legitimate statement
UNCHANGED-WILDCARD	Flagged for explicit justification — never silently accepted

Two design details that mattered in practice:

Feature detection is by predicate, not by an action list. The console grants GetBrowserSession, UpdateBrowserStream, s3files:ClientMount and similar. A hardcoded list drifts out of date the moment AWS adds an action, and the first attempt here matched none of the real names. The matchers key on each feature’s vocabulary instead.

The model ARNs come from the inference profile itself. A cross-Region profile requires both the profile ARN *and* the regional foundation-model ARNs it can route to — granting only the profile produces AccessDenied at invoke time. Rather than assume which regions, the script reads them from the live profile. This one routes to **three** regions; a naive us-* guess would have granted four, and the fourth would have been an unused permission.

2. Roles reviewed

Figures are the **rebuild, verified state** of 19 September, taken from `iam/iam_after.json`. Role names carry that session’s generated suffixes; the first submission’s roles (`-fpraf`, `_pcpa3`, `...1789761291328`) appear in `iam/session1-2026-09-18/` and in the quoted diffs in §3.

Role	Purpose	Statements removed	Statements narrowed	Wildcards surviving
onBedrockAgentCoreHarnessDefaultServiceRoleExecution —	invoke model, apply guardrail, invoke gateway, write logs	8	3	14
onBedrockAgentCoreGatewayDefaultServiceRoleExecution —	retrieve from the knowledge base	5	2	1
onBedrockAgentCoreKnowledgeBaseServiceRoleExecution —	read S3, invoke the embedding model	1	0	2

Totals: 14 statements removed, 5 narrowed, and 17 wildcard resources surviving (7 bare, 10 partial) — every one carrying a register entry. Statements widened: 0.

The knowledge-base role’s single removal is the whole `MarketplaceOperationsFromBedrockFor3pModels` statement. The first submission stripped only `aws-marketplace:Unsubscribe` from it and kept `Subscribe/ViewSubscriptions` on `*`; testing showed Titan embedding needs none of them, so the statement is gone (§4.2).

Both wildcard kinds are counted. The first submission’s table counted only bare `Resource: "*"`, which is how it reported the gateway role at zero while three partial wildcards remained; `22_iam_audit.py` now counts them separately and fails if any lacks a justification.

Five of those removals were not part of the original narrowing design. They were statements the first submission kept and labelled “loose”, which ablation then showed had no caller at all — see §4.2.

A finding about where the permissions actually live

The project brief implies the over-permission is in inline policies. **It is not.** The console-generated AgentCore roles put almost everything in **customer-managed attached policies**:

Role	Inline	Attached	Where the wildcards were
Harness	1	2	All 6 in the attached <code>...HarnessExecutionPolicy_le77v</code>
Gateway	0	2	Both in attached policies
Knowledge Base	0	3	Both in attached policies

This matters operationally: narrowing them requires `CreatePolicyVersion(SetAsDefault=True)`, not `PutRolePolicy`. A hardening script that only handles inline policies runs successfully, reports changes, and **leaves every real over-permission in place**. The first version of this script did exactly that.

3. Changes applied

How to read this section. It documents the narrowing pass statement by statement, and the quoted policy documents are the first submission’s — that is where each change was first derived, and the identifiers (`-fpraf`, `_pcpa3`, policy suffixes, the guardrail and gateway IDs) are from that session. The logic is unchanged in the rebuild; only the generated names differ.

Two changes described below were **superseded** by the rebuild, and each says so where it appears: AllowBedrockApplyGuardrailForKnowledgeBase on the gateway role and the marketplace statement on the knowledge-base role were not narrowed in the end — they were removed outright, after ablation showed neither had a caller (§4.2). The current state of every statement is §4 and iam/iam_after.json, both generated from the deployed policies.

3.1 Harness execution role

NARROWED — BedrockModelInvocation (*the headline change*)

Before:

```
"Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
"Resource": [
  "arn:aws:bedrock:*::foundation-model/*",
  "arn:aws:bedrock:us-east-1:118924230273:*"
]
```

After:

```
"Resource": [
  "arn:aws:bedrock:us-east-1:118924230273:inference-profile/us.amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-east-1::foundation-model/amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-east-2::foundation-model/amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-west-2::foundation-model/amazon.nova-2-lite-v1:0"
]
```

Rationale. The second ARN in the original — arn:aws:bedrock:us-east-1:118924230273:* — is **every Bedrock resource in the account**: every model, guardrail, knowledge base and inference profile, including resources that do not exist yet. The harness invokes exactly one inference profile. Granting foundation-model/* would additionally let a compromised role invoke every model in the account, including far more capable and more expensive ones.

NARROWED — ApplyNorthstarGuardrail

Scoped to one guardrail **and its cross-Region profile across three regions**:

```
arn:aws:bedrock:us-east-1:118924230273:guardrail/6yxbopugn44t
arn:aws:bedrock:us-east-1:118924230273:guardrail-profile/us.guardrail.v1:0
arn:aws:bedrock:us-east-2:118924230273:guardrail-profile/us.guardrail.v1:0
arn:aws:bedrock:us-west-2:118924230273:guardrail-profile/us.guardrail.v1:0
```

Rationale, and a finding. ApplyGuardrail on * would let the role apply **any** guardrail in the account — including a permissive one, which defeats the control it is supposed to enforce.

The four ARNs rather than one is not padding. A STANDARD-tier guardrail runs through cross-Region inference, so authorisation covers the guardrail *and the profile* — and **the profile ARN is regional, following request routing**. Granting only us-east-1 produced an intermittent failure when a call routed to us-east-2:

```
not authorized to perform: bedrock:ApplyGuardrail on resource:
arn:aws:bedrock:us-east-2:118924230273:guardrail-profile/us.guardrail.v1:0
```

The regions are read from the model's inference profile — the guardrail is applied wherever inference runs, so those are exactly the regions needed. Still enumerated; no wildcard. The project brief's sample policy grants only the guardrail ARN and would fail on any STANDARD-tier guardrail.

CloudWatch Logs statements — ⚠️ SEE ERRATUM §0

Only one of the four logging statements was genuinely narrowed. The other three were widened, and all four gained write access to the model-invocation log group. Full detail, root cause and the code fix are in §0; this section is left in place rather than rewritten so the error and its correction sit side by side.

Statement	Intended	Actually applied
CloudWatchLogsDescribeGroups	narrow from log-group:*	✅ narrowed as intended
CloudWatchLogsGroup	keep runtimes/*	❌ widened to bedrock-agentcore/*
CloudWatchLogsStream	keep runtimes/*:log-stream:*	❌ widened to bedrock-agentcore/*
CloudWatchLogsPutResourcePolicy	keep harness_NorthstarAssist-*	❌ widened to bedrock-agentcore/*

The valid part of the rationale. logs:DescribeLogGroups on log-group:* let the agent’s own role enumerate every log group in the account, and narrowing that was correct. The irony is not lost: the statement immediately after it in the same policy reasons about anti-forensics while the change being made handed the agent write access to its own audit log.

REMOVED — 7 statements, 28 permissions, all for features that are off

Statement	Actions	Why removed
AgentCoreBrowserDefault	7	Browser tool is off on the harness
AgentCoreCodeInterpreterDefault	5	Code Interpreter is off
AgentCoreMemory	5	Memory is disabled — no events to read or write
EFSClientAccess	2	No EFS mount configured
EFSDescribe	2	As above
S3FilesClientAccess	3	No S3 file-system mount configured
S3FilesDescribe	2	As above

Rationale. A permission with no caller is pure attack surface. These are exactly what a compromised role reaches for: StartBrowserSession gives outbound web access under the agent’s identity; InvokeCodeInterpreter gives arbitrary code execution; the file-system mounts give persistent read/write storage. None is needed to answer a policy question, and each was granted by default.

3.2 Gateway service role — 2 bare wildcards to 0

REMOVED — AllowBedrockAgenticRetrieveFromKnowledgeBase, AllowBedrockRerankForKnowledgeBase

Both granted bedrock:AgenticRetrieveStream / bedrock:Rerank on "*".

Rationale. The gateway target is configured for **Standard retrieval**, not Agentic retrieval. Neither action has a caller in this deployment, and both were granted on every resource.

Correction on the claim. An earlier draft said this took the role to “zero wildcards.” That was wrong, and it was wrong because the counter behind it only matched a bare Resource: "*". Removing those two statements took the role to **zero unscoped wildcards**; three *partial* wildcards remain and are justified in §4:

- bedrock-agentcore:GetConfigurationBundleVersion on configuration-bundle/*
- bedrock:GetInferenceProfile on inference-profile/*
- bedrock-agentcore:GetGateway on gateway/northstar-assist-gateway-1jkdshz4iy/*

The third is scoped to this gateway and is genuinely tight. The first two are not, and are recorded as such.

NARROWED — AllowBedrockInvokeModelForKnowledgeBase

From foundation-model/* plus inference-profile/*, provisioned-model/* and custom-model/* — to the Titan embedding ARN only.

Rationale. The gateway touches models solely for retrieval-side embedding. Granting it the generation model would let a compromised retrieval path run inference the design never intended. This is why the script resolves targets **per role** rather than globally: “the model” means different things to the harness and to the gateway.

NARROWED, then REMOVED — AllowBedrockApplyGuardrailForKnowledgeBase

Superseded. The narrowing below is what the first submission applied. In the rebuild this statement was **removed entirely**: the guardrail is applied by the *harness*, through its own scoped `NorthstarAssistApplyGuardrail` inline policy, and the gateway role’s copy had no caller. Removal was verified with a must-block prompt as well as a benign one, confirming the guardrail still intervenes end to end (§4.2).

From `guardrail/*` to the specific guardrail.

3.3 Knowledge base service role

Already well scoped by the console — `bedrock:InvokeModel` on the exact Titan ARN, `s3:ListBucket` and `s3:GetObject` on the exact bucket. One change needed.

ACTION-REMOVED, then statement REMOVED — aws-marketplace:Unsubscribe

Superseded. The first submission stripped `Unsubscribe` and kept `Subscribe` and `ViewSubscriptions` on `""`. In the rebuild the **whole statement was removed**: Titan Text Embeddings v2 is an AWS first-party model needing no Marketplace agreement, and a real ingestion job with a planted document indexed cleanly without the grant (`scanned=31 new=1 failed=0`) — see §4.2.

Before: `["aws-marketplace:Subscribe", "ViewSubscriptions", "Unsubscribe"] on ""` **After:** `["aws-marketplace:Subscribe", "ViewSubscriptions"] on ""`

Rationale. `Unsubscribe` on `*` lets this role **cancel foundation-model subscriptions for the entire account** — an availability attack against every AI workload in the account, not just this one. The knowledge base role needs to read and create subscriptions in order to use a model; it never needs to revoke one.

On why `Subscribe` and `ViewSubscriptions` were kept, rather than the whole statement removed. This is the weakest justification in the document and is flagged as such. The knowledge base uses **Titan Embeddings V2**, an AWS first-party model that needs no Marketplace agreement, so on the evidence available this statement has no caller at all and the stronger move is to remove it entirely — as was done for `Browser`, `Code Interpreter` and the rest. It was kept because the Marketplace interaction sits inside the managed Knowledge Base service rather than in code I could inspect, and removing a permission a managed service might take on first use risked breaking ingestion with the environment already torn down and no way to re-verify. **Recommended: remove the statement and re-run a sync to confirm.** Listed in §7.

A causal claim in an earlier draft was overstated and is withdrawn. That draft said this statement “explains” why Anthropic models failed — the harness role lacking marketplace permissions while the KB role held them. The `invoke` error does name `aws-marketplace:ViewSubscriptions`, `aws-marketplace:Subscribe` against the harness role, so the role genuinely lacks them. But `evidence/model_access.json` shows `agreementAvailability.status = NOT_AVAILABLE` for all three Anthropic models, which is an **account-level** entitlement state, not a role-policy one. Whether granting the harness role those actions would have let it auto-subscribe was never tested, and if no agreement is available to create then no role policy would have helped. The accurate statement is: **the role lacks the permission, and the account lacks the agreement; the account-level state is the more likely binding constraint, and the role-policy theory is untested.**

4. Surviving wildcards — every one accounted for

The hardened roles retain **17 wildcard resources** across 15 statements: 7 bare `Resource: ""` and 10 partial. Every one carries an entry in `iam/wildcard-register.json`, and `22_iam_audit.py` **fails** on any wildcard that does not — a missing

entry is a failure, not a default pass. That is the property the first submission lacked: its counter matched only bare "*", so seven partial wildcards were never counted.

Two classifications are permitted for a surviving wildcard:

- **UNAVOIDABLE** — the action does not support resource-level permissions. AWS rejects any Resource but "*"; the grant cannot be scoped by policy at all.
- **SCOPED** — the * is bounded to a resource path this system owns, or to a field that cannot be pinned, rather than being open across the account.

Worth being precise about what SCOPED does *not* claim. Two of these reach further than “one named resource”: `/aws/bedrock-agentcore/runtimes/*` covers every AgentCore runtime log group in the account, and `EcrManagedImagePull` wildcards the **account** field. Both are console defaults the narrowing pass deliberately left at their original scope — narrowing the first is precisely what produced the original regression — and each register entry states its own actual reach rather than inheriting the class definition.

A third class, **NECESSARY** — broader than ideal but proven required — is supported by the register and requires an evidence pointer to an ablation run. **No statement needed it.** Every candidate for it was removed outright instead (§4.2).

Role	Statement	Resource(s)	Class
gateway	GetGateway	arn:aws:bedrock-agentcore:us-east-1:118924230273:gateway/northstar-assist-gateway-lhirtedegn/*	SCOPED
harness	AgentCoreWorkloadIdentity	arn:aws:bedrock-agentcore:us-east-1:118924230273:workload-identity-directory/default/workload-identity/harness_NorthstarAssist-*	SCOPED
harness	BedrockMantleCallWithBearerToken	*	UNAVOIDABLE
harness	CloudWatchLogsDescribeLogGroups	arn:aws:logs:us-east-1:118924230273:log-group:/aws/bedrock-agentcore/runtimes/*arn:aws:logs:us-east-1:118924230273:log-group:/aws/bedrock-agentcore/runtimes/*:log-stream:*	SCOPED
harness	CloudWatchLogsGroup	arn:aws:logs:us-east-1:118924230273:log-group:/aws/bedrock-agentcore/runtimes/*arn:aws:logs:us-east-1:118924230273:log-group:/aws/bedrock-agentcore/runtimes/*:log-stream:*	SCOPED
harness	CloudWatchLogsPutResourcePolicy	arn:aws:logs:us-east-1:118924230273:log-group:/aws/bedrock-agentcore/runtimes/harness_NorthstarAssist-*	SCOPED
harness	CloudWatchLogsStream	arn:aws:logs:us-east-1:118924230273:log-group:/aws/bedrock-agentcore/runtimes/*:log-stream:*	SCOPED
harness	CloudWatchMetricsPublish	*	UNAVOIDABLE
harness	EcrManagedImagePull	arn:aws:ecr:us-east-1:*:repository/harness-*	SCOPED
harness	EcrManagedImageToken	*	UNAVOIDABLE
harness	EcrPublicTokenAccess	*	UNAVOIDABLE
harness	StsForEcrPublicPull	*	UNAVOIDABLE
harness	XRayTracingAccess	*	UNAVOIDABLE
kb	CloudWatchWritePermissionStatement	*	UNAVOIDABLE
kb	S3GetObjectStatement	arn:aws:s3:::northstar-assist-kb-41071520/*	SCOPED

Full justifications, one per statement, are in `iam/wildcard-register.json`; `iam/iam_after.md` renders them alongside this table.

4.1 The account-field wildcard, stated plainly

`EcrManagedImagePull` reads `arn:aws:ecr:us-east-1*:repository/harness-*`. The repository prefix is pinned; the remaining `*` is the **account** field, because AWS publishes AgentCore harness runtime images from an AWS-owned account whose ID is not documented. Pinning it would need that ID. The grant is read-only image pull — it cannot push or delete — but it is the one surviving wildcard whose breadth is imposed by missing documentation rather than by the API, and it is classified SCOPED on the strength of the repository prefix alone.

4.2 Wildcards removed rather than justified

Five statements were candidates for a NECESSARY justification. Instead of writing one, each was **removed from the live role and the system retested**. A permission the system does not use cannot break it by being taken away, so a failure after removal would have been strong evidence of necessity. None failed.

Statement	Wildcard removed	Probe	Result
<code>BedrockMantleInference</code>	<code>arn:aws:bedrock-mantle:us-east-1:118924230273:</code>	retrieval + answer	removed — system unaffected
<code>GetConfigurationBundleVersion</code>	<code>arn:aws:bedrock-agentcore:us-east-1:118924230273:configuration-bundle/*</code>	retrieval + answer	removed — system unaffected
<code>AllowBedrockGetInferenceProfileForKnowledgeBase</code>	<code>arn:aws:bedrock:us-east-1:118924230273:inference-profile/</code>	retrieval + answer	removed — system unaffected
<code>AllowBedrockApplyGuardrailForKnowledgeBase</code>	<code>arn:aws:bedrock:us-east-1:118924230273:guardrail/</code>	retrieval + answer	removed — system unaffected
<code>MarketplaceOperationsFromBedrockFor3pModels</code>		ingestion job	removed — system unaffected

The marketplace test is the one worth reading closely. `aws-marketplace:Subscribe` and `ViewSubscriptions` on `*` sat on the knowledge-base role, and the knowledge-base role is used at **ingestion** time, not query time — so a smoke test cannot exercise it at all. The first attempt ran a sync over an already-indexed corpus and reported `scanned=30 new=0 modified=0`: it completed having embedded nothing, which proves nothing about a permission checked at embedding time. The test was rerun with a document planted first, giving `scanned=31 new=1 failed=0` — a document genuinely embedded with Titan v2 while the marketplace grant was absent. The planted document was then deleted and the corpus reverified at 30 of 30 retrievable.

This also closes out a claim withdrawn after the first review. I said then that the harness role’s missing marketplace permissions might explain the Anthropic model failures, and had to withdraw it as untested. I still cannot assert that causation — `agreementAvailability` is an account-level state — but I can now say that Titan embedding does not need the grant, because it was removed and a document was embedded anyway.

4.3 One wildcard justified by shape, not by experiment

`BedrockMantleCallWithBearerToken` keeps `Resource: "*"` on the resource-shape argument alone: bearer-token exchange names no resource, and bedrock-mantle publishes no resource-level ARNs for it. **It was not ablated** — and its sibling `BedrockMantleInference` was, and proved unnecessary. That is reason enough to say so plainly rather than let the UNAVOIDABLE label imply more testing than was done. The same holds for the other six UNAVOIDABLE statements: they are justified by the API’s constraints, not by removal tests. Where a wildcard *could* be removed, it was.

4.4 Reading a removal result honestly

A **failure** after removal is strong evidence of necessity. A **pass** is weaker: an AgentCore runtime caches its role credentials, so a permission withdrawn seconds earlier may still be in force. Every removal here was therefore confirmed a second time — applied permanently, left for 120 seconds, then retested with three smoke runs **and** a must-block prompt to confirm the guardrail still intervened. Verifying a removed ApplyGuardrail grant with a prompt that never trips the guardrail would have repeated the original mistake: testing the path where the permission is not used.

One limit, stated rather than glossed: the five removals were confirmed **together**, not one at a time, so no single statement has an isolated post-removal confirmation. Each was also tested individually during the ablation pass, where every one was restored afterwards. Both the individual results and the combined final state are evidenced; the combination of the two is not.

5. Verification — two kinds, and why one was not enough

5.1 Behavioural verification

Over-tightening an agent’s role produces a failure mode worse than leaving it loose: **the agent keeps answering, but without retrieval**. Answers look plausible, are entirely ungrounded, and nothing in the response indicates it.

So --apply is paired with --verify, which after IAM propagation runs one live invocation and checks four things: no error, the guardrail did not block a benign question, the Retrieve tool actually fired, and chunks came back non-zero. **On failure it automatically restores every original policy** — inline and managed — replaying from iam/before/.

```
[13:58:53] OK applied ...HarnessExecutionPolicy_647sw [managed]
[13:58:53] OK applied ...GatewayKBAccessProd_28A9EB [managed]
[13:58:53] OK applied ...S3PolicyForKnowledgeBase_y7cmq [managed]
[13:58:53] ==> waiting 15s for IAM propagation before verifying
[13:59:15] OK harness still answers with retrieval after narrowing
```

5.2 Why that was not enough

This check passed in the first submission too — while three logging statements were being widened. It had to. Widening a permission never breaks anything, so a test that asks “does the agent still work?” cannot detect one. Behavioural verification is necessary and structurally blind to the failure that actually occurred.

The correct question is not *is it still running* but *does the policy now grant less than it did*. That is a question about the policy, and it has to be asked of the policy.

5.3 Policy-level verification

scripts/22_iam_audit.py reads the after-state — from iam/after/ offline, or from IAM directly with --live — and for every statement, matched by Sid:

1. **Nothing widened.** Every resource in the narrowed statement must be covered by at least one resource in the original, and no action may appear that was not there before. Both comparisons are wildcard-aware, so xray:PutTraceSegments is not reported as new when the original said xray:Put*.
2. **Every widening is declared.** Hardening is not purely subtractive — granting bedrock:ApplyGuardrail on the cross-Region guardrail-profile ARNs is a genuine new grant without which a STANDARD-tier guardrail 403s. Such a change is not exempted from the check; it must be named, resource by resource, in the register’s intentional_grants, with a reason. An undeclared widening still fails.
3. **Every surviving wildcard is accounted for**, per §4.

Verdict on the rebuilt environment, read back from live IAM:

```
statements 37   narrowed 5   removed 14   unchanged 18   widened 0
wildcards 17 (7 bare, 10 partial)   unjustified 0
AUDIT PASS - nothing widened, every wildcard justified
```

5.4 Reproducing this without AWS

The audit runs **offline by default** against the committed evidence, so the verdict above can be reproduced from a clone with no AWS credentials:

```
python submission/scripts/22_iam_audit.py
python submission/tests/test_no_widening.py
```

The first re-derives the PASS from iam/before/ versus iam/after/. The second replays every captured console policy through the real narrowing pass and asserts nothing widens — 16 policies, 74 statements, 0 widened, covering this rebuild and the first submission’s originals together.

And because a detector that has never caught anything is only a claim:

```
python submission/scripts/22_iam_audit.py      --before iam/session1-2026-09-18/before      --after iam/session1-2026-09-18/after --expect-fail
```

Pointed at the first submission’s preserved evidence, the auditor exits 0 *because it fails* — reporting the three logging widenings, the S3 widening, and the unjustified wildcards. The detector is shown catching the real defect, not a synthetic one.

Rollback remains available. All original policy documents are preserved and --restore replays them, including managed policies via a new default version. The 5-version IAM cap is handled by pruning the oldest non-default version first.

6. What an attacker gains — before versus after

With the original console-generated roles, a principal able to assume the harness execution role could:

- Invoke **any** Bedrock model in the account, including more capable and costlier ones — and any guardrail, knowledge base or inference profile, present or future, via `arn:aws:bedrock:us-east-1:118924230273:*`
- Apply **any** guardrail, including a permissive one, neutralising the control
- Start **browser sessions** — outbound web access under the agent’s identity
- **Execute arbitrary code** via the Code Interpreter
- Mount **EFS and S3 file systems** — persistent read/write storage
- Read and write **agent memory**
- **Enumerate every log group** in the account

And a principal holding the knowledge base role could **cancel foundation-model subscriptions account-wide**, disabling every AI workload in the account.

After narrowing, the same principal can:

- Invoke **one** inference profile and the three regional model ARNs it routes to
- Apply **one** guardrail, via its own cross-Region profile
- Invoke **one** gateway
- Write to **this system’s** log groups
- Publish metrics and traces (no resource-level permissions exist)
- Pull the harness runtime image

The gateway role can read and retrieve from **one** knowledge base and embed with **one** model. The knowledge base role can read **one** bucket, embed with **one** model, and can no longer revoke anything.

Blast radius for a compromise of the agent’s identity goes from “the account’s entire AI estate, plus code execution and outbound network” to “read 30 internal documents.” That is what least privilege actually bought here — not a tidier policy document.

7. Remaining recommendations

1. Add a `cloudwatch:namespace` condition to both PutMetricData statements, restricting them to NorthstarAssist/Security. This is the only surviving wildcard that can be meaningfully constrained.

2. **Add a permissions boundary** to all three service roles, so future console operations cannot re-widen them. These roles were generated permissive by default and a console edit could regenerate them that way.
3. **Alert on PutRolePolicy / CreatePolicyVersion / AttachRolePolicy** against these three roles via CloudTrail. Narrowing is only durable if re-widening is visible.
4. **Populate runtimeUserId** so invocation logs attribute activity to an employee rather than to the shared execution role — see the monitoring plan §1.5.
5. **Create a break-glass decommissioning role** with credentials independent of the operating ones. During this assessment a credential revocation left live infrastructure running and unreachable; the containment path should not share fate with the operating path.

Items 6-9, raised after the first review — now closed

#	Item	Status
6	Re-apply the corrected logging statements and re-verify	Done. Rebuilt 19 Sep; CloudWatchLogsStream and CloudWatchLogsPutResourcePolicy kept at their original tighter scope via KEPT-WOULD-WIDEN, and the harness role holds no grant on /northstar-assist/model-invocations.
7	Make verification check permissions, not behaviour	Done. 22_iam_audit.py asserts on the policy; tests/test_no_widening.py replays the captured policies offline.
8	Remove MarketplaceOperationsFromBedrockFor3p and confirm ingestion	Done. Models Removed; a planted document embedded cleanly with Titan v2 (scanned=31 new=1 failed=0), corpus reverified 30 of 30.
9	Scope the three loose partial wildcards	Done, by removal. inference-profile/*, configuration-bundle/* and bedrock-mantle:...:* were each removed outright after ablation showed no caller — better than the narrowing originally proposed.

Still open

10. **Confirm each removal individually against a cold runtime.** The five removals were verified together. Each was also tested individually during ablation, but with the statement restored afterwards; no single statement has an isolated post-removal confirmation. A rebuild that removes one at a time would close this.
11. **Pin the ECR account field** in EcrManagedImagePull (§4.1) if AWS ever documents the account publishing AgentCore harness runtime images.
12. **Re-test guardrail v2.** The 38.5% benign over-block rate measured in the first submission has diagnosed causes and drafted fixes that remain untested; this rebuild was scoped to the IAM finding and did not re-run the 29-test suite.

Evidence

Artefact	Location
Audit verdict, machine-readable	iam/iam_after.json
Audit verdict, rendered	iam/iam_after.md
Wildcard register	iam/wildcard-register.json
Ablation experiments and results	iam/ablation-results.json
First submission, preserved unedited	iam/session1-2026-09-18/
Original policy documents (8)	iam/before/
Narrowed policy documents	iam/after/
Unified diffs, per policy	iam/after/*.diff
Machine-readable change log	evidence/iam/iam_changes.json
Full before/after capture	evidence/iam/iam_before.json, iam_after.json
Generator output	evidence/iam/generated-change-log.md
Narrowing implementation	scripts/20_harden_iam.py